

Chapter 6: Database Design Using the E-R Model

- Design Process
- Modeling
- Constraints
- E-R Diagram
- Design Issues
- Weak Entity Sets
- Extended E-R Features
- Design of the Bank Database
- Reduction to Relation Schemas
- Database Design
- UML



- The initial phase of database design is to characterize fully the data needs of the prospective database users.
- Next, the designer chooses a data model and, by applying the concepts of the chosen data model, translates these requirements into a conceptual schema of the database.
- A fully developed conceptual schema also indicates the functional requirements of the enterprise. In a “specification of functional requirements”, users describe the kinds of operations (or transactions) that will be performed on the data.



The process of moving from an abstract data model to the implementation of the database proceeds in two final design phases.

■ Logical Design – Deciding on the database schema. Database design requires that we find a “good” collection of relation schemas.

● Business decision – What attributes should we record in the database?

● Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?

■ Physical Design – Deciding on the physical layout of the database

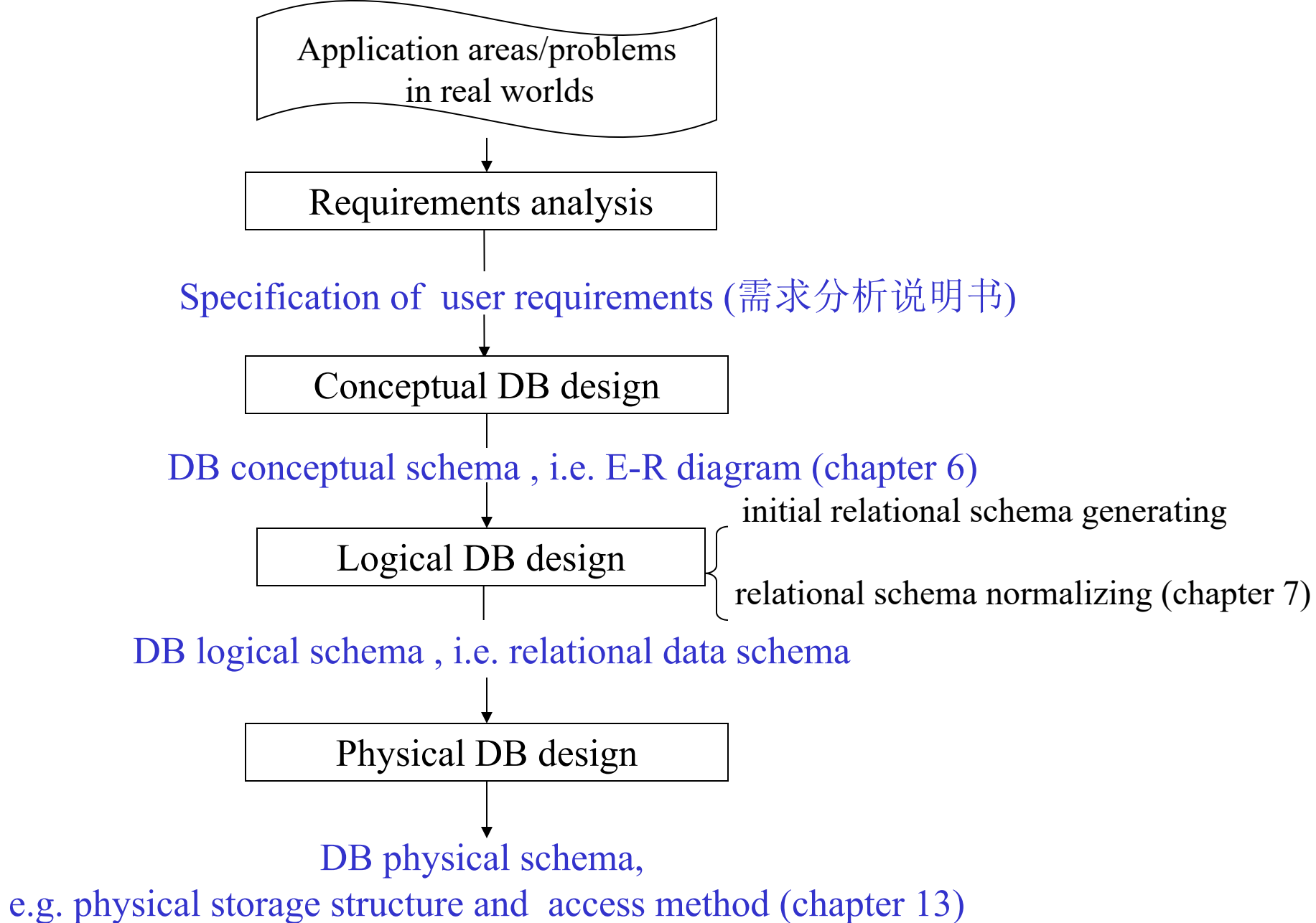


- Entity Relationship Model (covered in this chapter)
 - Models an enterprise as a collection of *entities* and *relationships*
 - ▶ Entity: a “thing” or “object” in the enterprise that is distinguishable from other objects
 - Described by a set of *attributes*
 - ▶ Relationship: an association among several entities
 - Represented diagrammatically by an *entity-relationship diagram*:
- Normalization Theory (Chapter 7)
 - Formalize what designs are bad, and test for them

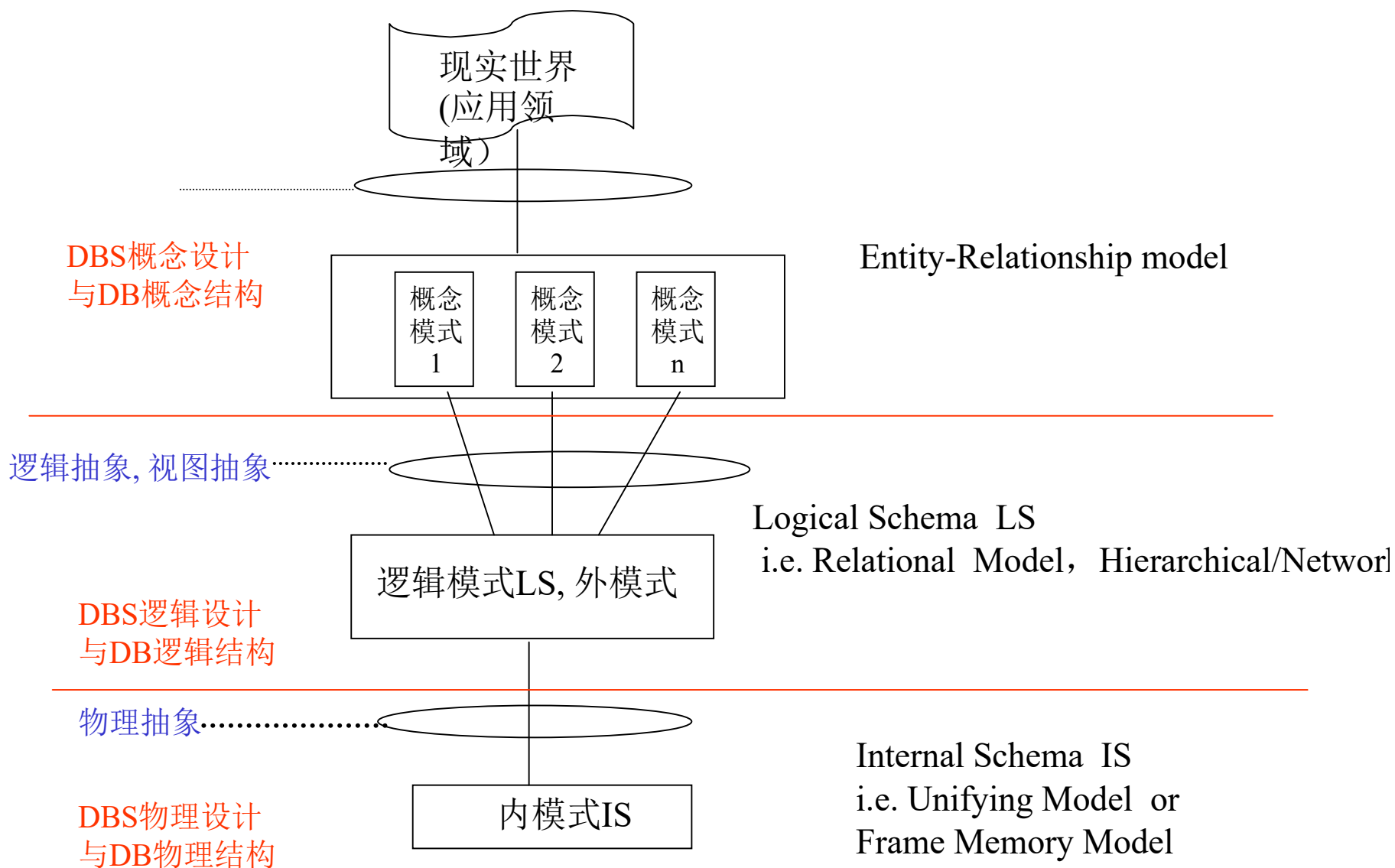


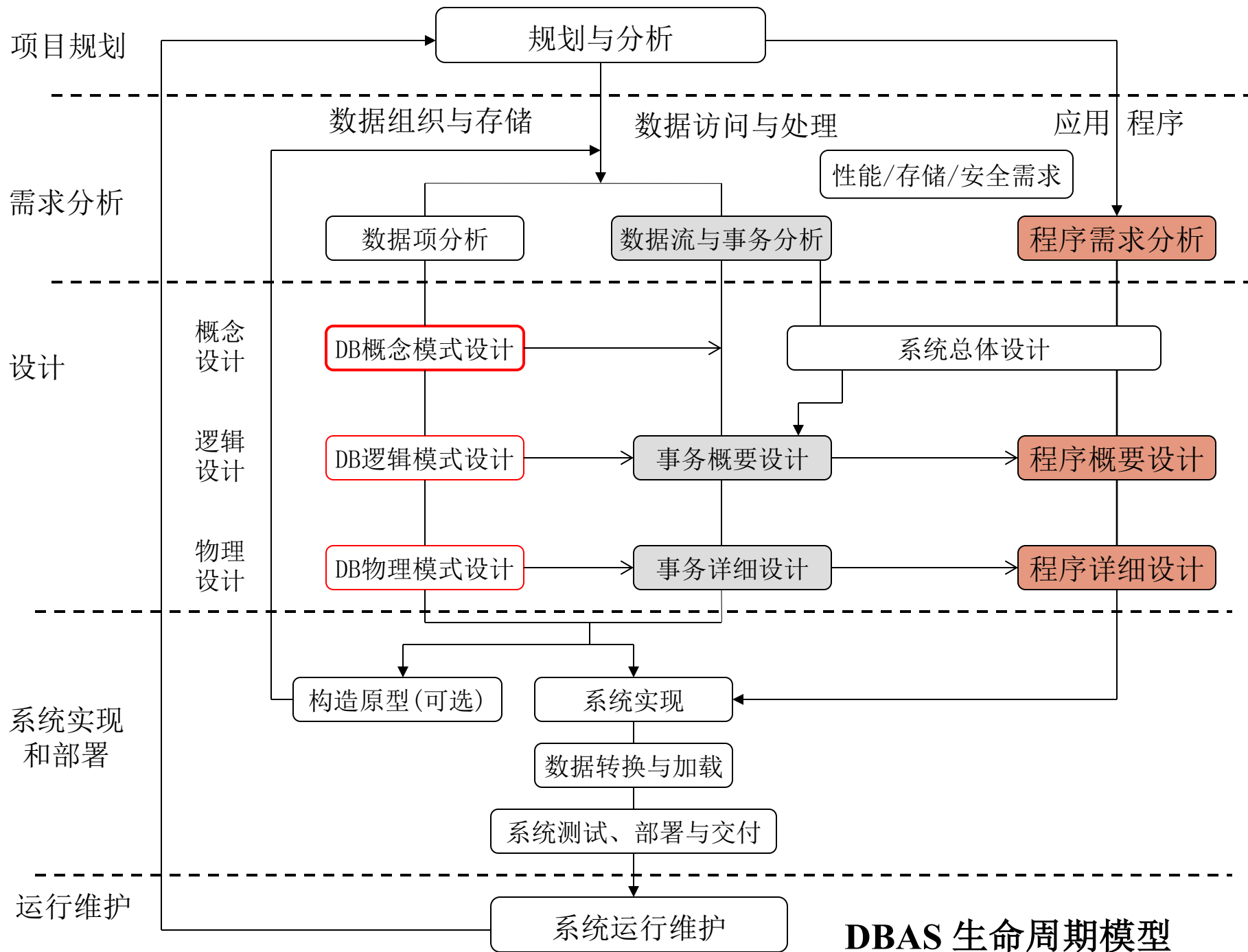
- Requirements Analysis;
- Conceptual Schema Design; (概念结构/模式设计)
- Logical schema design; (逻辑结构/模式设计)
- Physical Schema Design;
- Implementation;
- System Testing;
- Delivery & Maintenance





DB design phases





DBAS 生命周期模型

Outline of the ER Model

ER model -- Database Modeling

DataBase System Concepts

- The ER data model was developed to facilitate database design by allowing specification of an **enterprise schema** that represents the overall logical structure of a database.
- The ER model is very useful in mapping the meanings and interactions of real-world enterprises onto a conceptual schema. Because of this usefulness, many database-design tools draw on concepts from the ER model.
- The ER data model employs three basic concepts:
 - entity sets,
 - relationship sets,
 - attributes.
- The ER model also has an associated diagrammatic representation, the ER diagram, which can express the overall logical structure of a database graphically.



- An **entity** is an object that exists and is distinguishable from other objects.

- Example: specific person, company, event, plant

- An **entity set** is a set of entities of the same type that share the same properties.

- Example: set of all persons, companies, trees, holidays

- An entity is represented by a set of attributes; i.e., descriptive properties possessed by all members of an entity set.

- Example:

instructor = (ID, name, street, city, salary)

course = (course_id, title, credits)

- A subset of the attributes form a **primary key** of the entity set; i.e., uniquely identifying each member of the set.



Entity Sets -- *instructor* and *student*

instructor_ID instructor_name

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

instructor

student-ID student_name

98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

■ Attribute types:

- **Simple** and **composite** attributes.

- **Single-valued** and **multivalued** attributes

 - ▶ Example: multivalued attribute: *phone_numbers*

- **Derived** attributes

 - ▶ Can be computed from other attributes

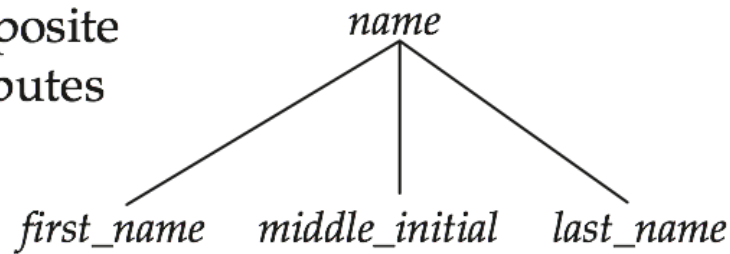
 - ▶ Example: age, given date_of_birth

■ **Domain** – the set of permitted values for each attribute

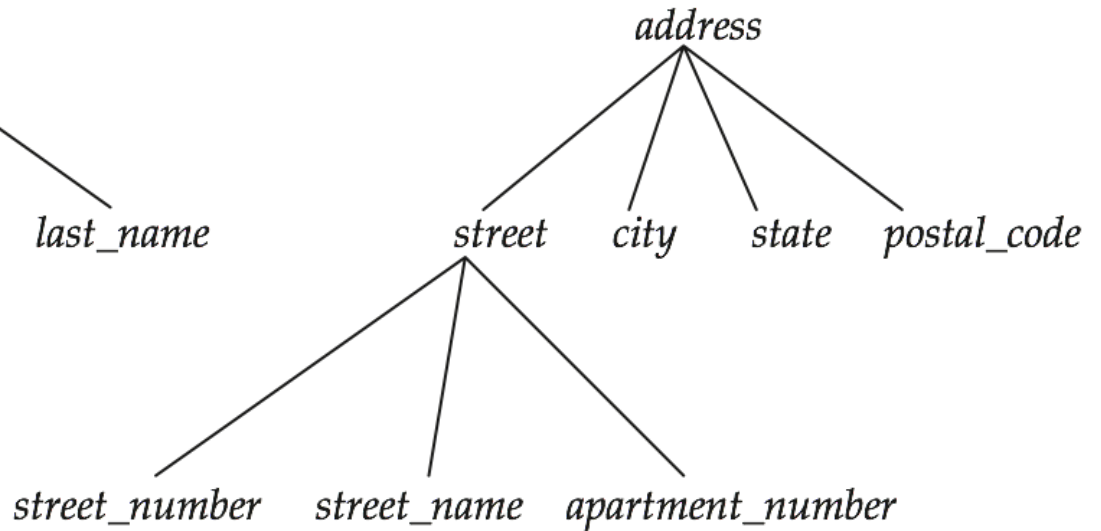


Composite Attributes

composite
attributes



component
attributes



- A **relationship** is an association among several entities

Example:

44553 (Peltier) advisor 22222 (Einstein)
student entity relationship set *instructor* entity

- A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

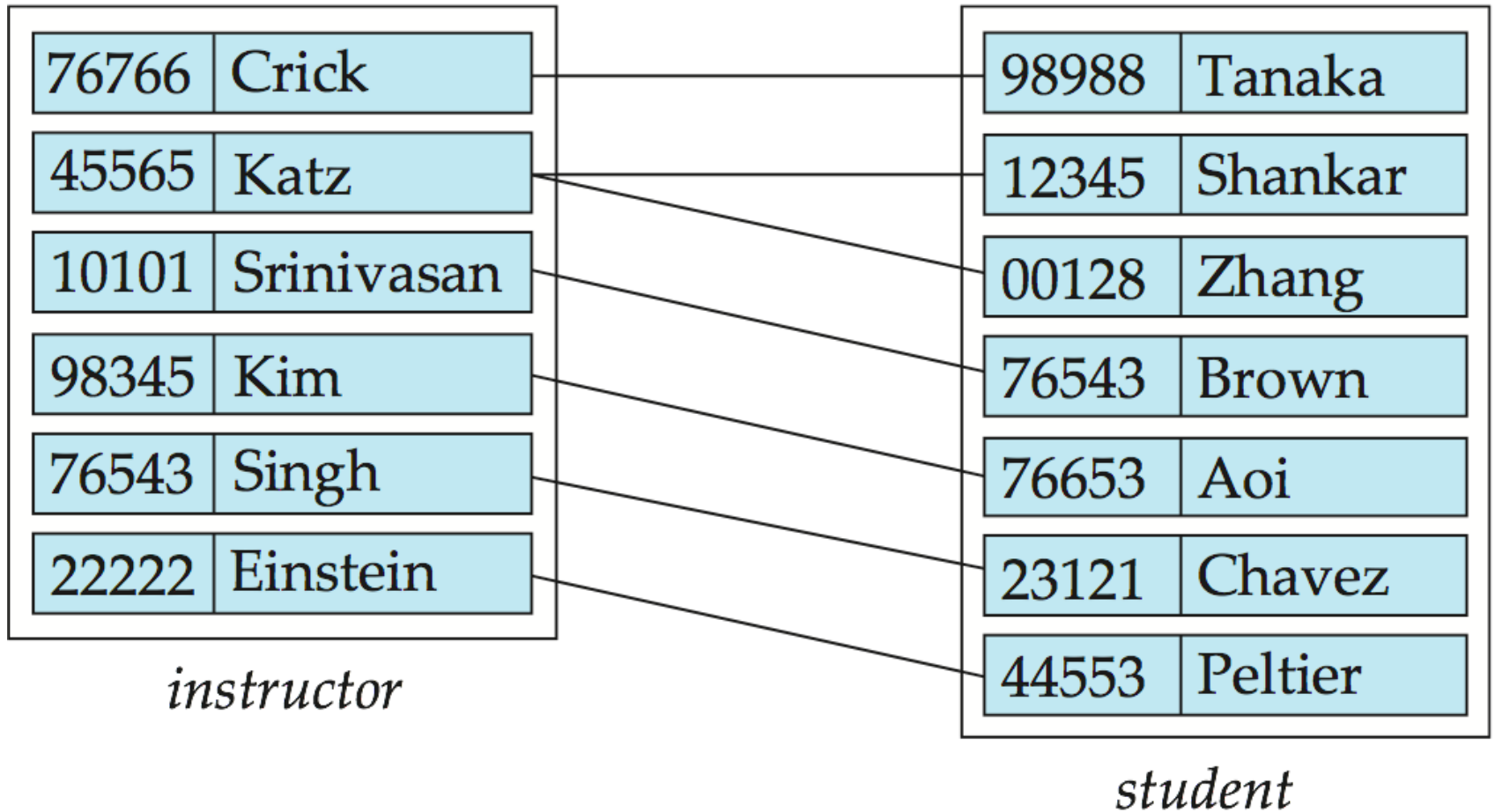
where $(e_1, e_2, \dots, e_n)$ is a relationship

- Example:

$$(44553, 22222) \in \text{advisor}$$

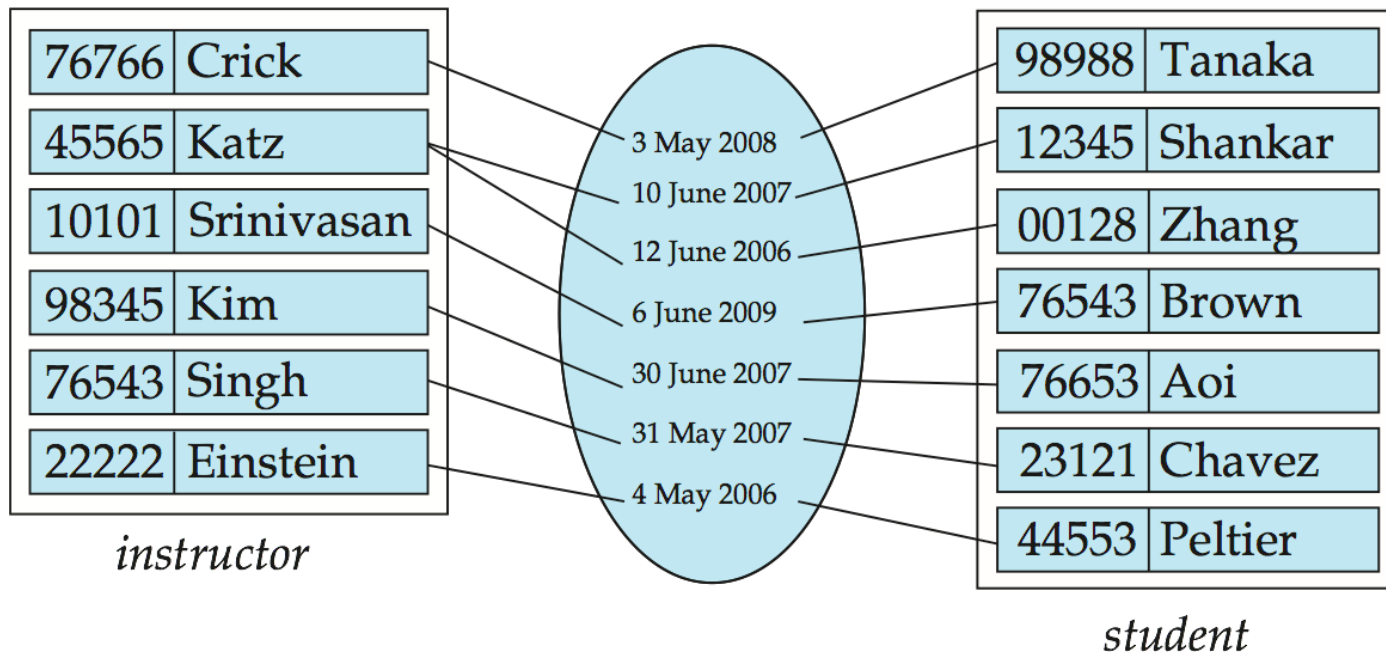


Relationship Set *advisor*



Relationship Sets (Cont.)

- An attribute can also be associated with a relationship set.
- For instance, the *advisor* relationship set between entity sets *instructor* and *student* may have the attribute *date* which tracks when the student started being associated with the advisor



■ Suppose we have entity sets:

● *instructor*, with attributes: *ID*, *name*, *dept_name*, *salary*

● *department*, with attributes: *dept_name*, *building*, *budget*

■ We model the fact that each instructor has an associated department using a relationship set *inst_dept*

■ The attribute *dept_name* appears in both entity sets. Since it is the primary key for the entity set *department*, it replicates information present in the relationship and is therefore redundant in the entity set *instructor* and needs to be removed.



Degree of a Relationship Set DataBase System Concepts

- binary relationship
 - involve two entity sets (or degree two).
 - most relationship sets in a database system are binary.
- Relationships between more than two entity sets are rare. Most relationships are binary. (More on this later.)
 - ▶ Example: *students* work on research *projects* under the guidance of an *instructor*.
 - ▶ relationship *proj_guide* is a ternary relationship between *instructor*, *student*, and *project*



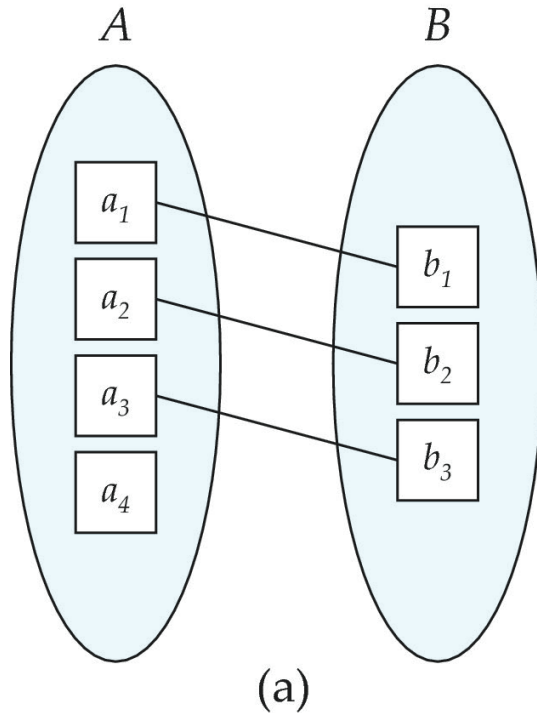
Mapping Cardinality Constraints

DataBase System Concepts

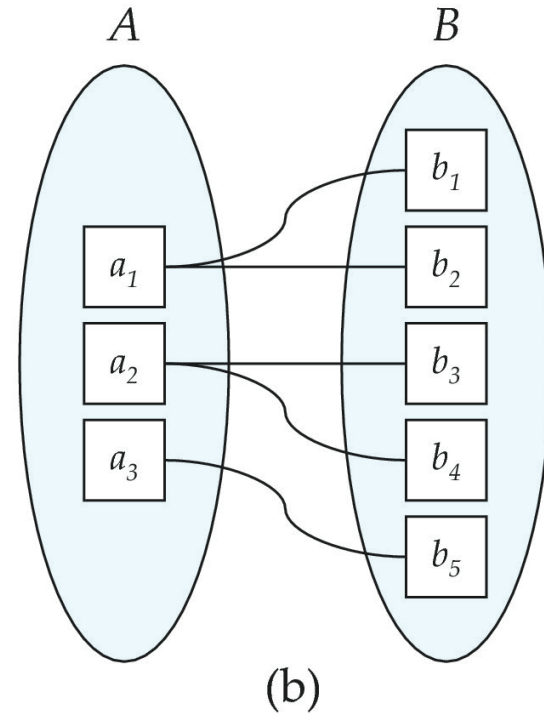
- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
 - One to one
 - One to many
 - Many to one
 - Many to many



Mapping Cardinalities



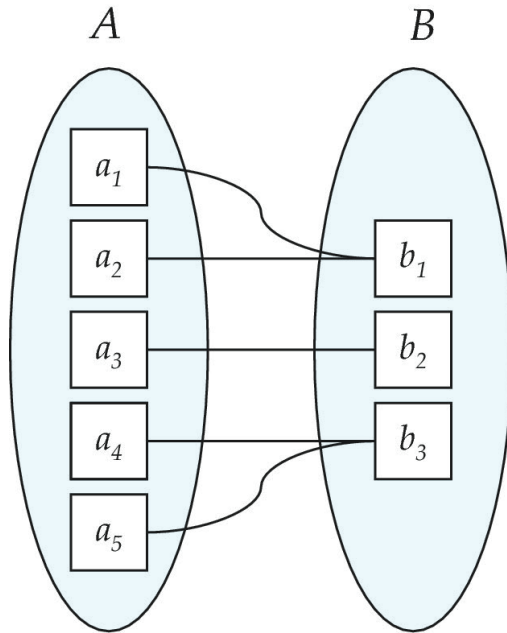
One to one



One to many

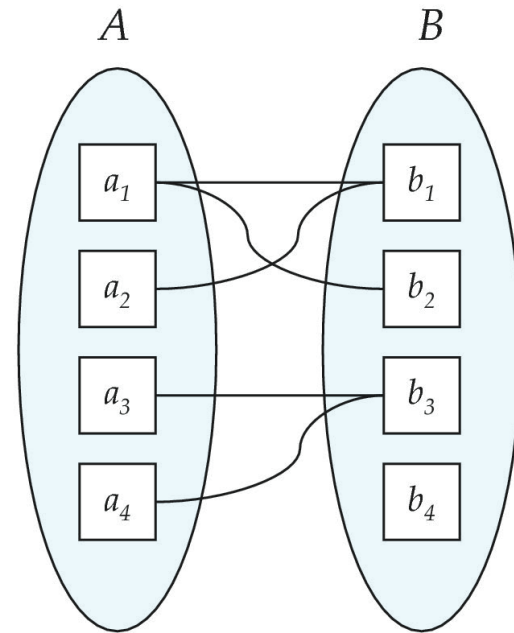
Note: Some elements in A and B may not be mapped to any elements in the other set

Mapping Cardinalities



(a)

Many to
one



(b)

Many to many

Note: Some elements in A and B may not be mapped to any elements in the other set

- Consider a *section* entity, which is uniquely identified by a *course_id*, *semester*, *year*, and *sec_id*.
- Clearly, section entities are related to course entities. Suppose we create a relationship set *sec_course* between entity sets *section* and *course*.
- Note that the information in *sec_course* is redundant, since *section* already has an attribute *course_id*, which identifies the course with which the section is related.
- One option to deal with this redundancy is to get rid of the relationship *sec_course*; however, by doing so the relationship between *section* and *course* becomes implicit in an attribute, which is not desirable.



Weak Entity Sets (Cont.)

DataBase System Concepts

- An alternative way to deal with this redundancy is to not store the attribute *course_id* in the *section* entity and to only store the remaining attributes *section_id*, *year*, and *semester*. However, the entity set *section* then does not have enough attributes to identify a particular *section* entity uniquely; although each *section* entity is distinct, sections for different courses may share the same *section_id*, *year*, and *semester*.
- To deal with this problem, we treat the relationship *sec_course* as a special relationship that provides extra information, in this case, the *course_id*, required to identify *section* entities uniquely.
- The notion of **weak entity set** formalizes the above intuition. A weak entity set is one whose existence is dependent on another entity, called its **identifying entity**; instead of associating a primary key with a weak entity, we use the identifying entity, along with extra attributes called **discriminator** to uniquely identify a weak entity. An entity set that is not a weak entity set is termed a **strong entity set**.

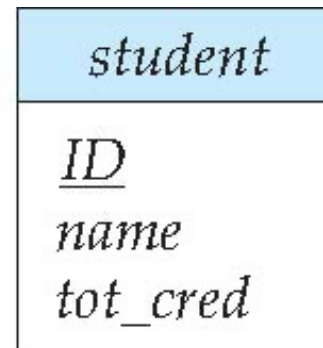
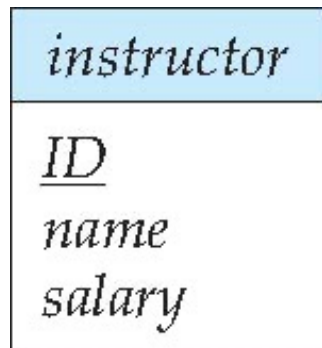


- Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be **existence dependent** on the identifying entity set. The identifying entity set is said to **own** the weak entity set that it identifies. The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**.
- Note that the relational schema we eventually create from the entity set *section* does have the attribute *course_id*, for reasons that will become clear later, even though we have dropped the attribute *course_id* from the entity set *section*.



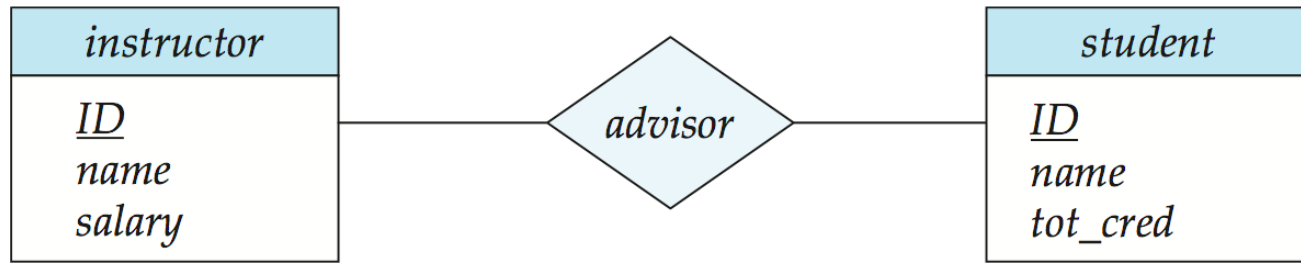
E-R Diagrams

- Entities can be represented graphically as follows:
 - Rectangles represent entity sets.
 - Attributes listed inside entity rectangle
 - Underline indicates primary key attributes



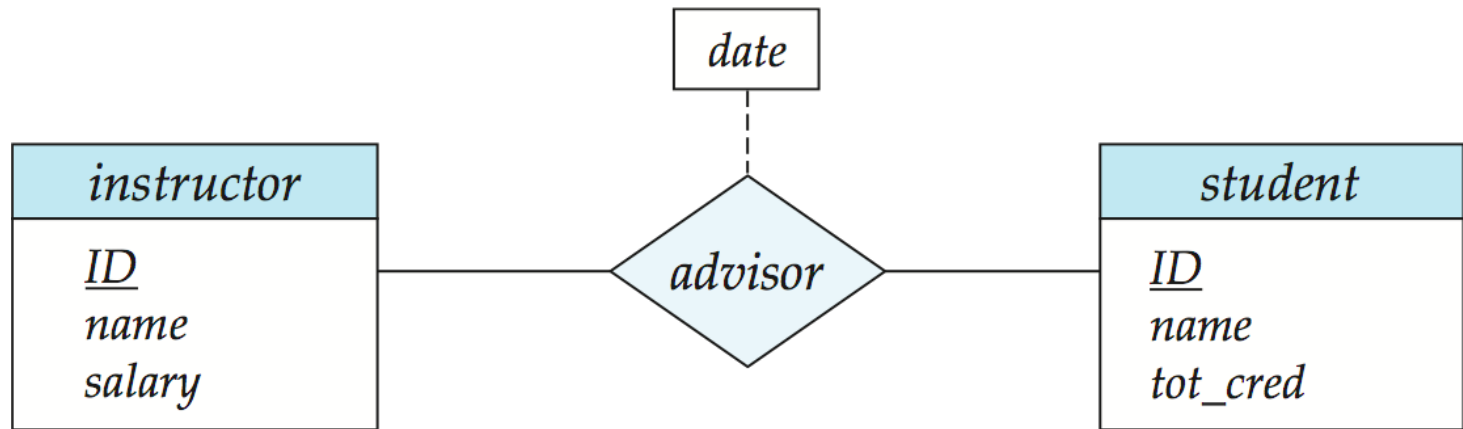
Relationship Sets

- Diamonds represent relationship sets.

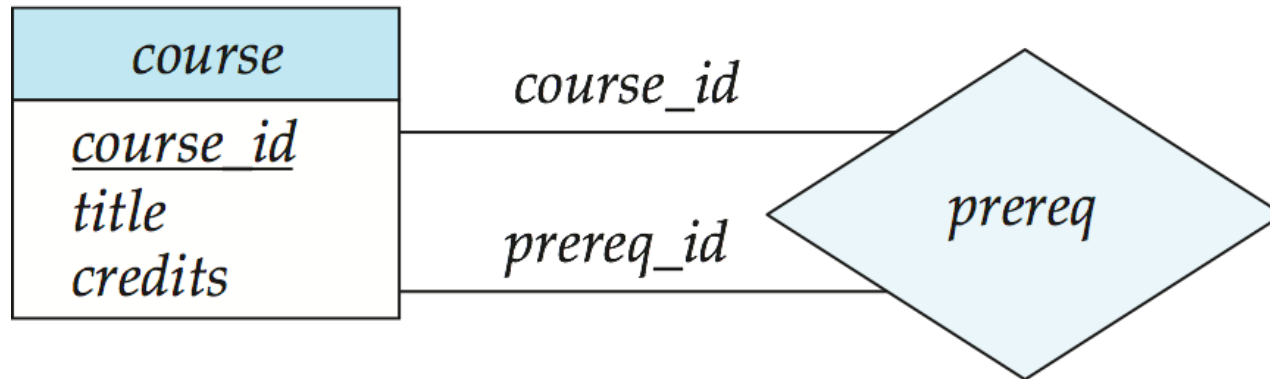


Relationship Sets with Attributes

DataBase System Concepts

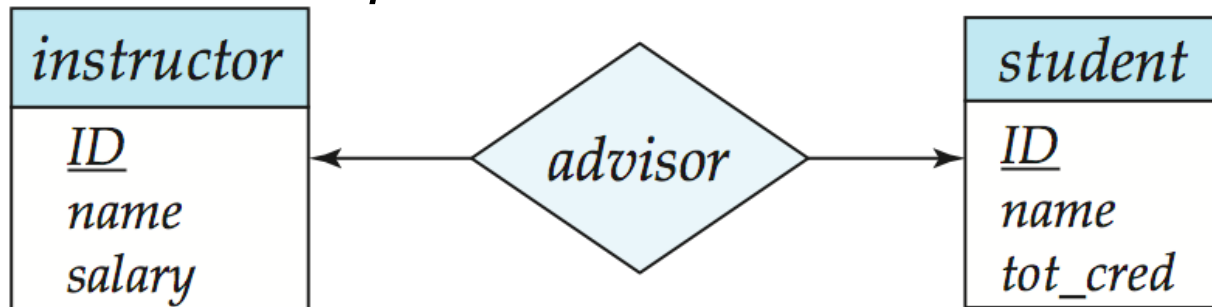


- Entity sets of a relationship need not be distinct
 - Each occurrence of an entity set plays a “role” in the relationship
- The labels “*course_id*” and “*prereq_id*” are called **roles**.



Cardinality Constraints

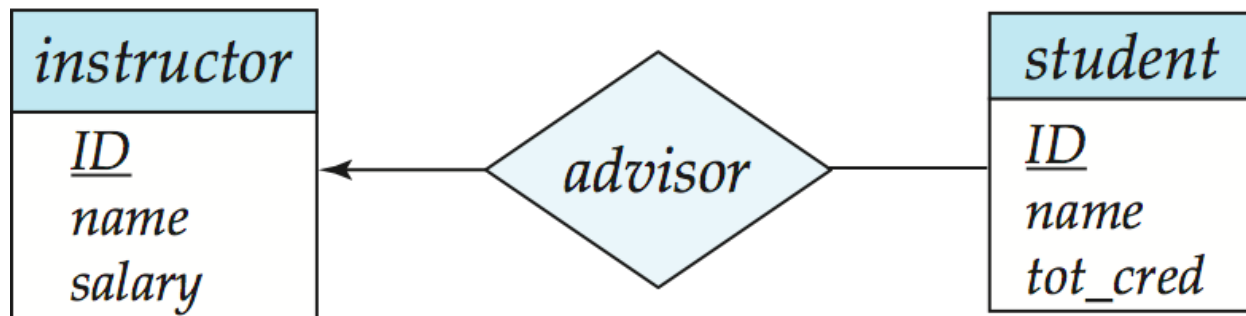
- We express cardinality constraints by drawing either a directed line ($\rightarrow$), signifying “one,” or an undirected line ($-$), signifying “many,” between the relationship set and the entity set.
- One-to-one relationship between an *instructor* and a *student* :
 - A student is associated with at most one *instructor* via the relationship *advisor*
 - A *student* is associated with at most one *department* via *stud_dept*



One-to-Many Relationship

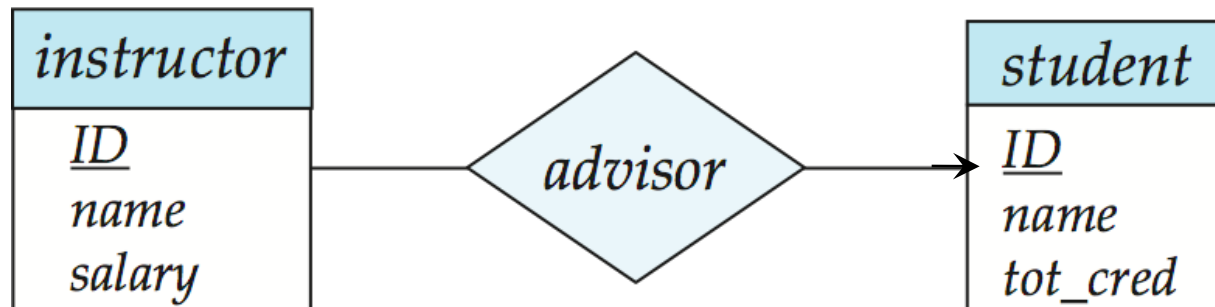
DataBase System Concepts

- one-to-many relationship between an *instructor* and a *student*
 - an instructor is associated with several (including 0) students via *advisor*
 - a student is associated with at most one instructor via *advisor*,



Many-to-One Relationships

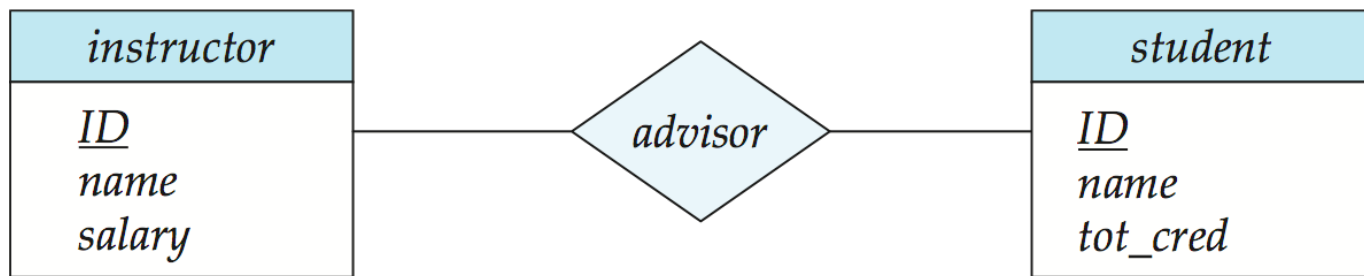
- In a many-to-one relationship between an *instructor* and a *student*,
 - an instructor is associated with at most one student via *advisor*,
 - and a student is associated with several (including 0) instructors via *advisor*



Many-to-Many Relationship

DataBase System Concepts

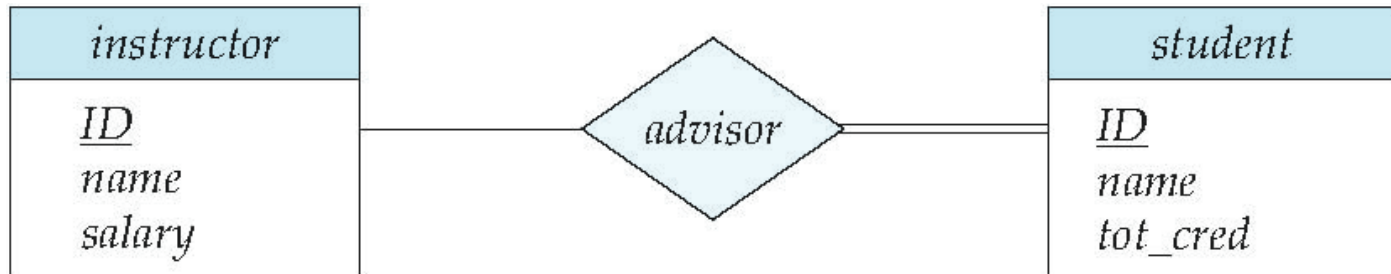
- An instructor is associated with several (possibly 0) students via *advisor*
- A student is associated with several (possibly 0) instructors via *advisor*



Total and Partial Participation

DataBase System Concepts

- Total participation (indicated by double line): every entity in the entity set participates in at least one relationship in the relationship set



participation of *student* in *advisor* relation is total

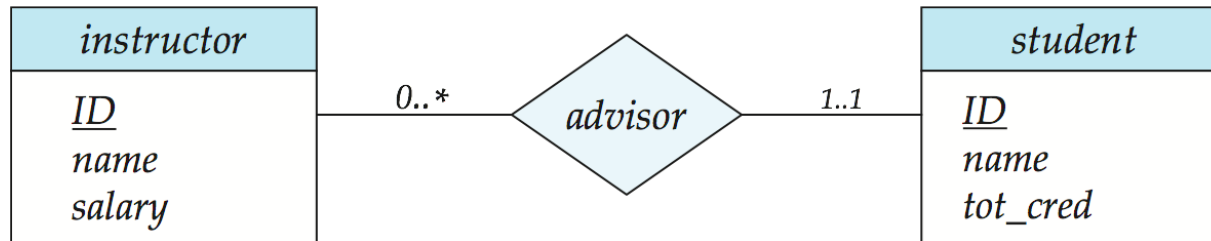
- ▶ every *student* must have an associated instructor

- Partial participation: some entities may not participate in any relationship in the relationship set
 - Example: participation of *instructor* in *advisor* is partial



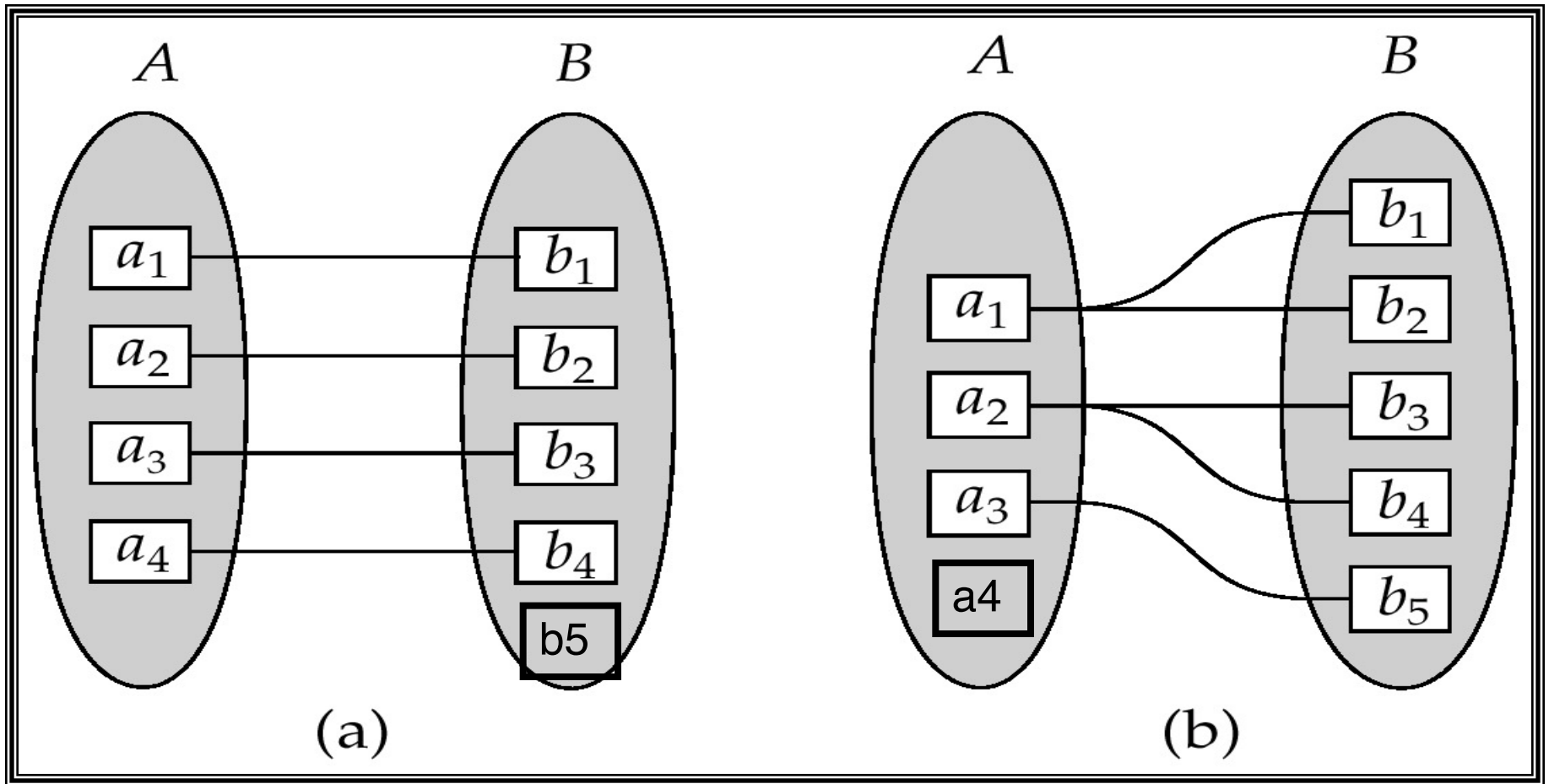
Notation for Expressing More Complex Constraints

- A line may have an associated minimum and maximum cardinality, shown in the form $l..h$, where l is the minimum and h the maximum cardinality
 - A minimum value of 1 indicates total participation.
 - A maximum value of 1 indicates that the entity participates in at most one relationship
 - A maximum value of * indicates no limit.



Instructor can advise 0 or more students. A student must have 1 advisor; cannot have multiple advisors





From A to B, one to one (1:1),

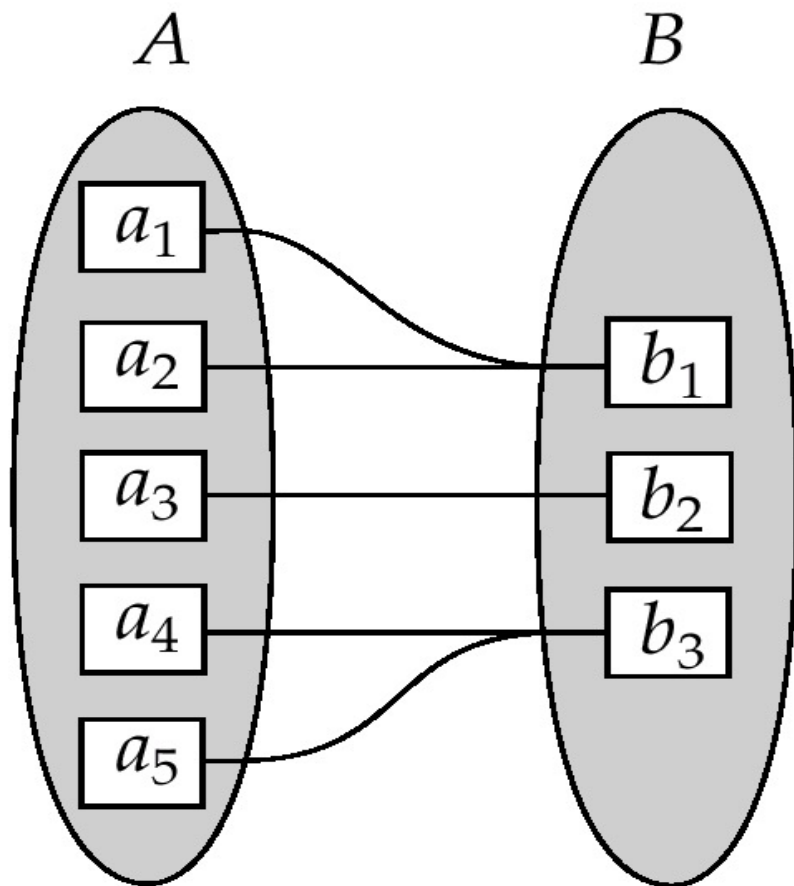
$$\langle l_A, h_A \rangle = \langle 1, 1 \rangle,$$

$$\langle l_B, h_B \rangle = \langle 0, 1 \rangle;$$

From A to B, one to many (1:2),

$$\langle l_A, h_A \rangle = \langle 0, 2 \rangle,$$

$$\langle l_B, h_B \rangle = \langle 1, 1 \rangle$$



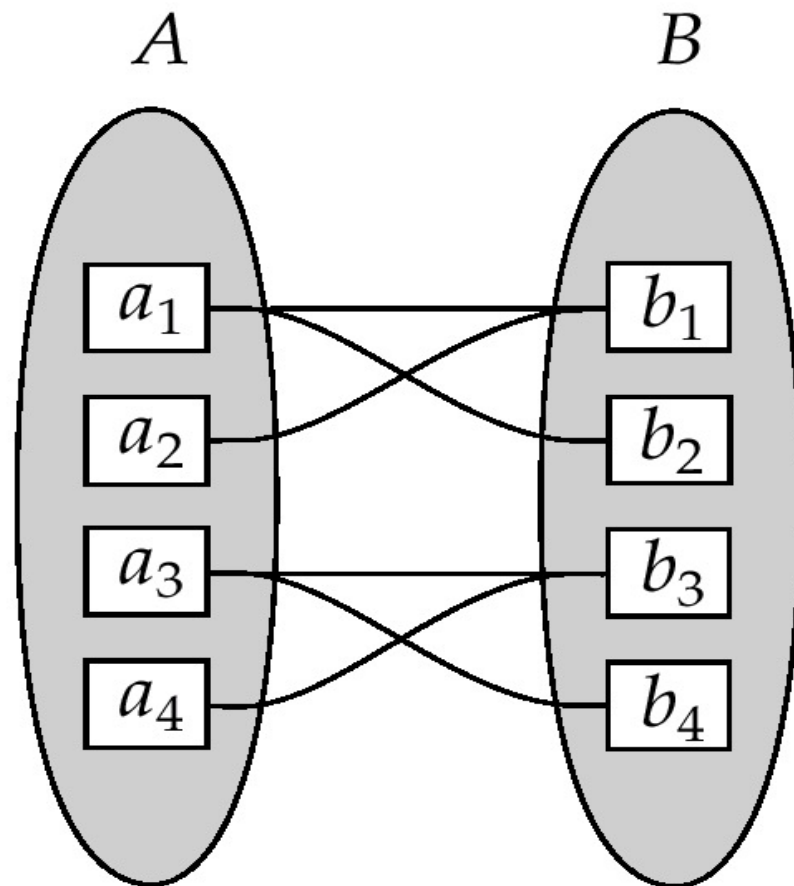
(a)

Form A to B, many to one

(2:1),

$$\langle l_A, h_A \rangle = \langle 1, 1 \rangle,$$

$$\langle l_B, h_B \rangle = \langle 1, 2 \rangle$$



(b)

Form A to B, many to

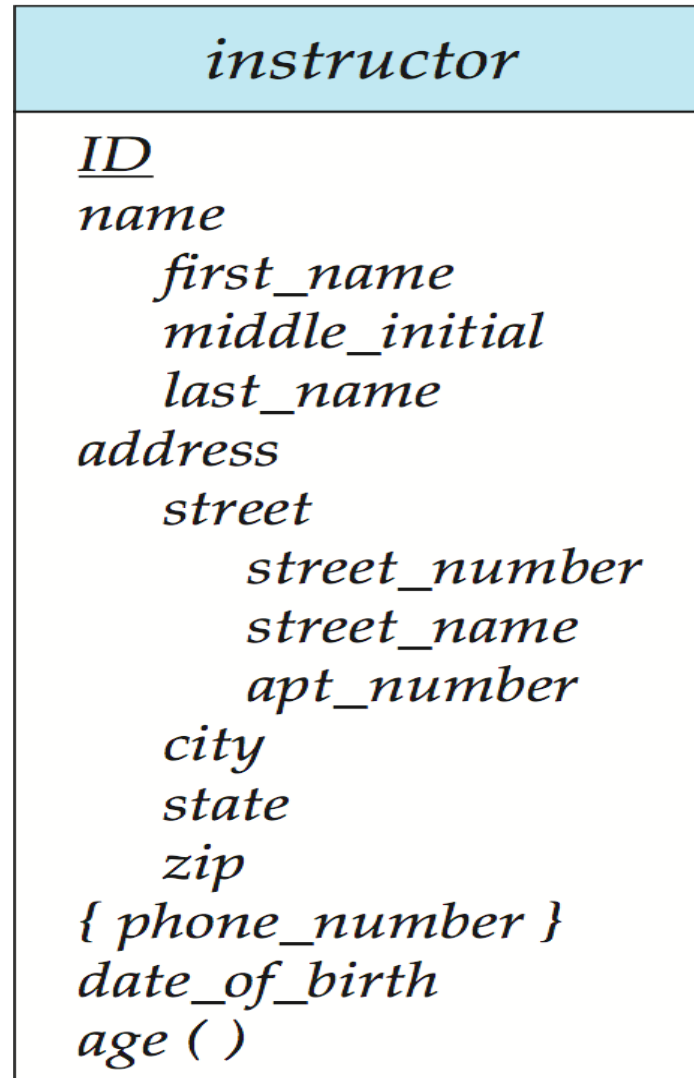
many (2:2)

$$\langle l_A, h_A \rangle = \langle 1, 2 \rangle$$

$$\langle l_B, h_B \rangle = \langle 1, 2 \rangle$$

Notation to Express Entity with Complex Attributes

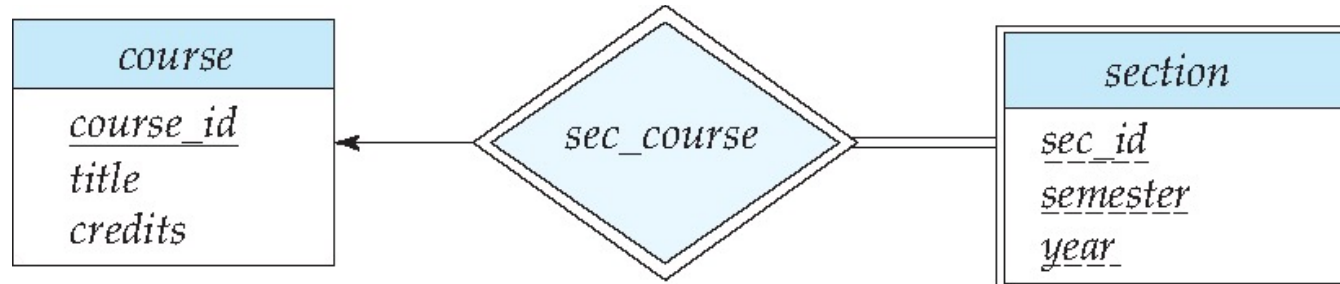
DataBase System Concepts



Expressing Weak Entity Sets

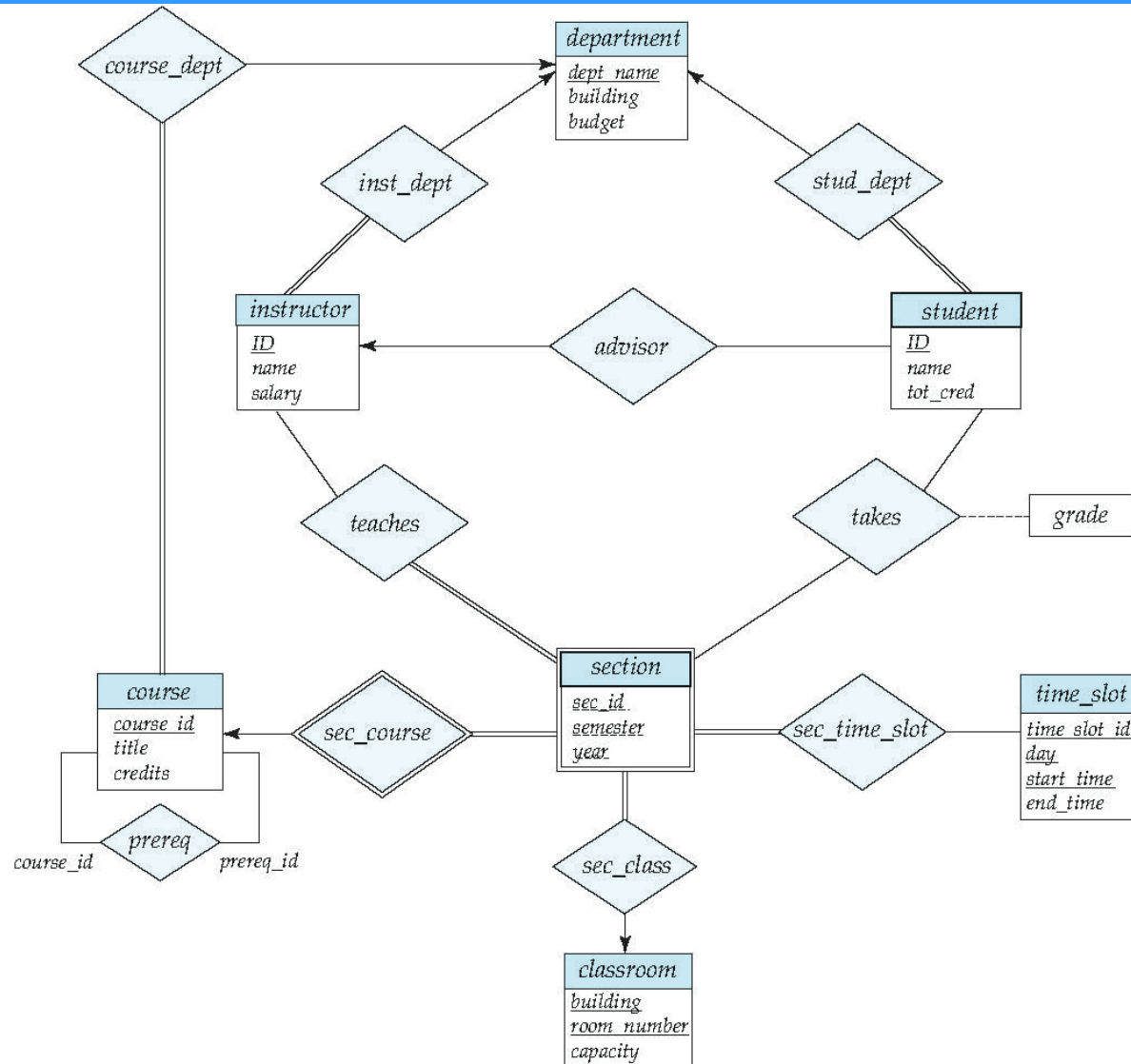
DataBase System Concepts

- In E-R diagrams, a weak entity set is depicted via a double rectangle.
- We underline the discriminator of a weak entity set with a dashed line.
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.
- Primary key for *section* – (*course_id*, *sec_id*, *semester*, *year*)



E-R Diagram for a University Enterprise

DataBase System Concepts



Ordering system example

- A database used in an ordering (订货/订购) system contains information about *customers*, *products* and *orders*. The following information is to be included:
 - for each *customer*
 - *customer-number*, *ship-to-addresses* (several per customer), *name*, *balance*, *discount*
 - for each *order* (订单)
 - *order number*, *customer number*, *ship-to address* (one per order), *date of ordering* (including *year*, *month*, and *day*), *products* (several kinds per order), and *quantity ordered of each product*



■ for each kind of *product*

product-number, name, manufacturing-plant, quantity on hand

■ Some constraints(i.e., integrity constraints) about this ordering system are as follows

- a customer may *make* one or more orders, and some customers may make no orders
- each order must be for one (and only one) customer
- each order must *contain* at least one kind of products
- a kind of product may appear in several orders, but some kinds of products may not be ordered by any customer.



- 1: Entity set ***Customer***, ***Order***, ***Product*** and their *attributes*. In ***Customer***, *ship-to addresses* is a multi-valued attribute, but in ***Order***, *ship-to addresses* is a single-valued attribute. In ***Order***, *date* is a composite attribute
- 2:
 - relationship set ***make*** between ***Customer*** and ***Order***
 - ***Customer*** partly participate ***make***, and ***Order*** totally participate ***make***
 - in relationship set ***make***, mapping cardinality from ***Customer*** to ***Order*** is one-to-many



■ *answer3*

- relationship set *contain* between *Order* and *Product*, and
- descriptive attributes *quantity of ordered* for relationship set *contain*
- *Product* partly participate *contain*, and *Order* totally participate *contain*
- In relationship set *contain*, mapping cardinality from *Order* to *Product* is many-to-many (or one-to-many)



- 1 设计实体集
- 2 设计联系集
- 3 设计属性，实体集属性，联系集属性

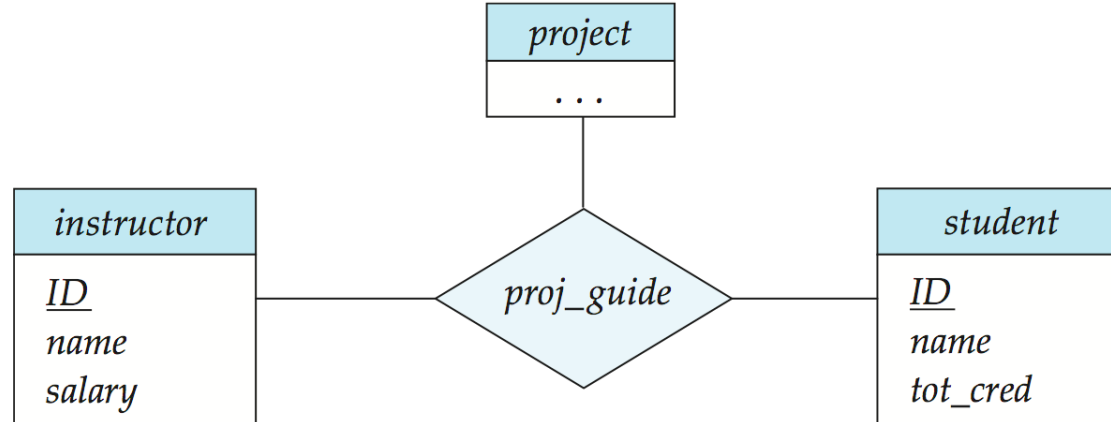


Advanced Topics

Non-binary Relationship Sets

DataBase System Concepts

- Most relationship sets are binary
- There are occasions when it is more convenient to represent relationships as non-binary.
- E-R Diagram with a Ternary Relationship



Cardinality Constraints on Ternary Relationship

- We allow at most one arrow out of a ternary (or greater degree) relationship to indicate a cardinality constraint
- For example, an arrow from *proj_guide* to *instructor* indicates each student has at most one guide for a project
- If there is more than one arrow, there are two ways of defining the meaning.
 - For example, a ternary relationship R between A , B and C with arrows to B and C could mean
 1. Each A entity is associated with a unique entity from B and C or
 2. Each pair of entities from (A, B) is associated with a unique C entity, and each pair (A, C) is associated with a unique B
 - Each alternative has been used in different formalisms
 - To avoid confusion we outlaw more than one arrow

Specialization

- Top-down design process; we designate sub-groupings within an entity set that are distinctive from other entities in the set.
- These sub-groupings become lower-level entity sets that have attributes or participate in relationships that do not apply to the higher-level entity set.
- Depicted by a *triangle* component labeled ISA (e.g., *instructor* “is a” *person*).
- **Attribute inheritance** – a lower-level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked.



- Constraint on whether or not entities may belong to more than one lower-level entity set within a single generalization.

- Disjoint (正交/不相交)

- an entity can belong to only one lower-level entity set

- Noted in E-R diagram by writing *disjoint* next to the ISA triangle

- Overlapping (重叠)

- an entity can belong to more than one lower-level entity set



Design Constraints on a Specialization/Generalization

- **Completeness constraint** -- specifies whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within a generalization.
 - **total**: an entity must belong to one of the lower-level entity sets
 - **partial**: an entity need not belong to one of the lower-level entity sets
- The *student* generalization is total: All student entities must be either graduate or undergraduate.

Disjoint , total Specialization/Generalization

Disjoint, partial Specialization/Generalization

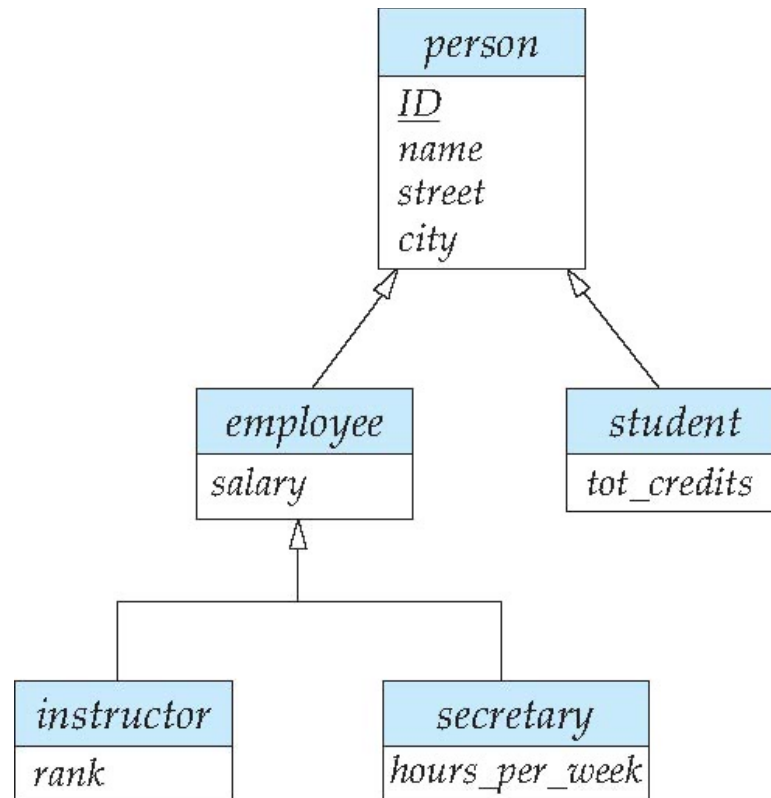
Overlapping, partial Specialization/Generalization

Overlapping, total Specialization/Generalization



Specialization Example

- **Overlapping** – *employee* and *student*
- **Disjoint** – *instructor* and *secretary*
- Total and partial

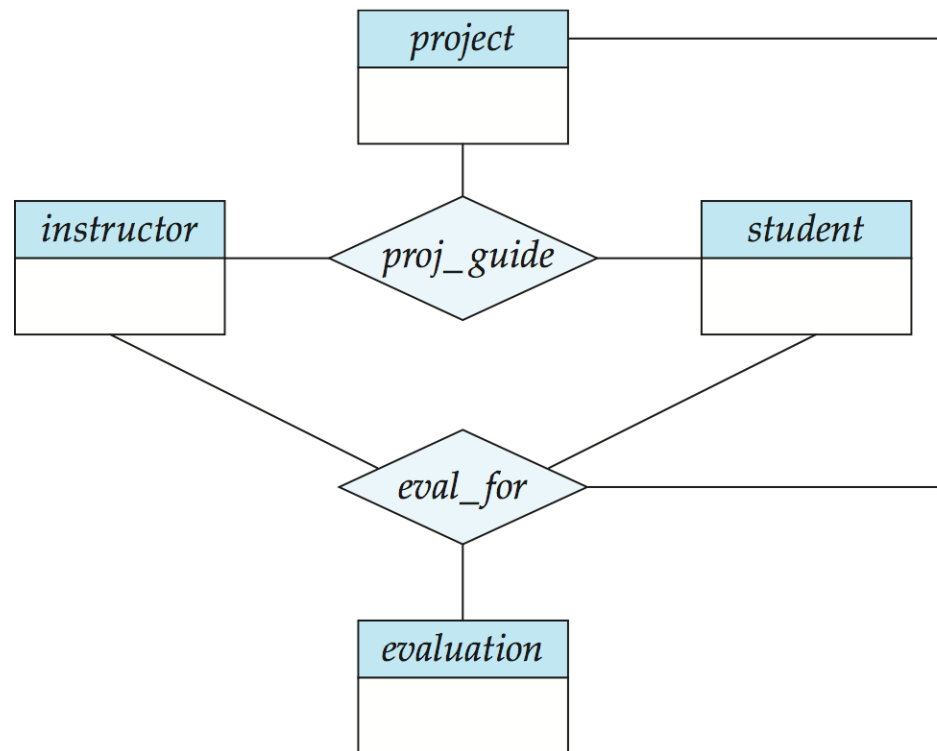


- **A bottom-up design process** – combine a number of entity sets that share the same features into a higher-level entity set.
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way.
- The terms specialization and generalization are used interchangeably.



Aggregation

- Consider the ternary relationship *proj_guide*, which we saw earlier
- Suppose we want to record evaluations of a student by a guide on a project



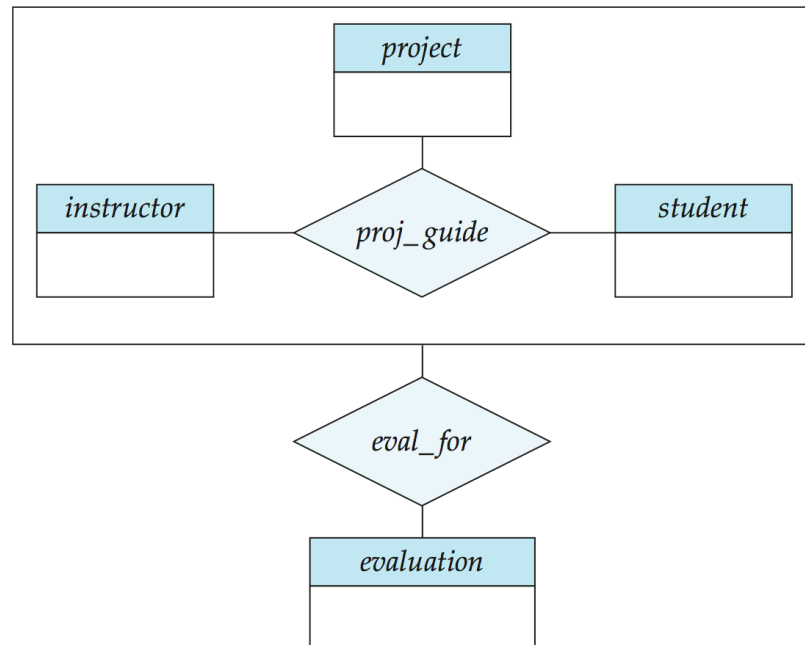
Aggregation (Cont.)

- Relationship sets *eval_for* and *proj_guide* represent overlapping information
 - Every *eval_for* relationship corresponds to a *proj_guide* relationship
 - However, some *proj_guide* relationships may not correspond to any *eval_for* relationships
 - ▶ So we can't discard the *proj_guide* relationship
- Eliminate this redundancy via *aggregation*
 - Treat relationship as an abstract entity
 - Allows relationships between relationships
 - Abstraction of relationship into new entity



Aggregation (Cont.)

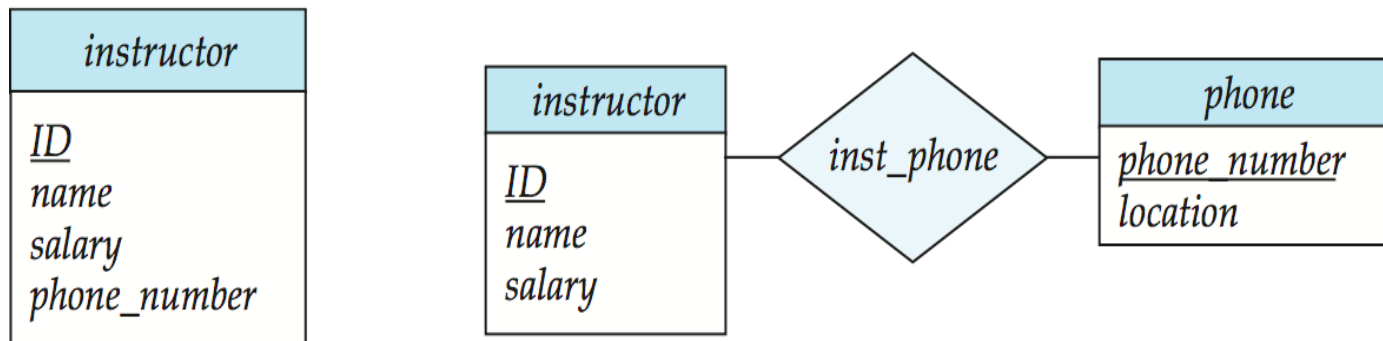
- Eliminate this redundancy via *aggregation* without introducing redundancy, the following diagram represents:
 - A student is guided by a particular instructor on a particular project
 - A student, instructor, project combination may have an associated evaluation



Design Issues

Entities vs. Attributes

■ Use of entity sets vs. attributes



■ Use of phone as an entity allows extra information about phone numbers (plus multiple phone numbers)

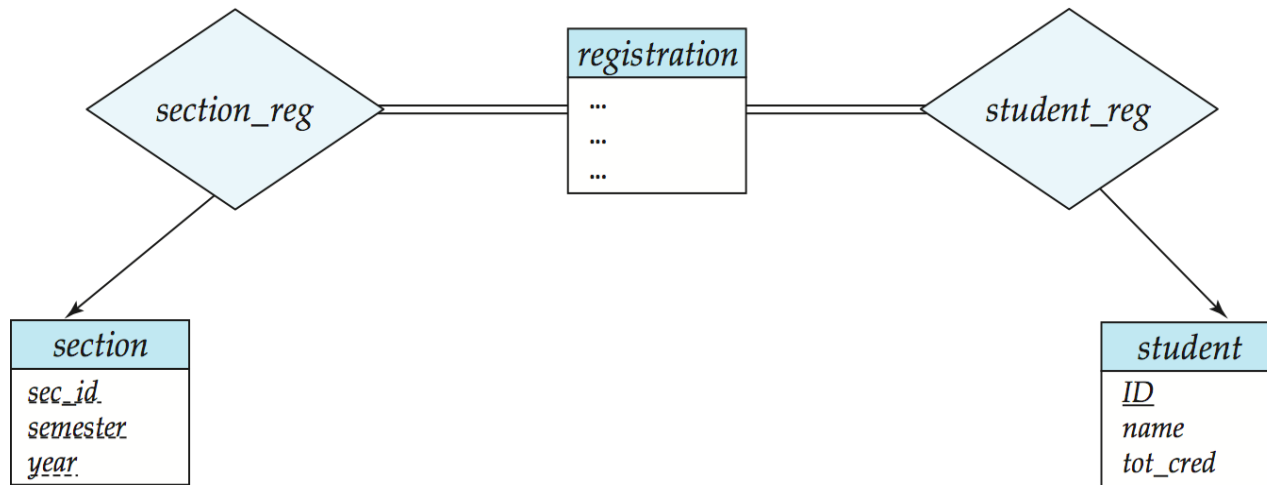


Entities vs. Relationship sets

DataBase System Concepts

■ Use of entity sets vs. relationship sets

Possible guideline is to designate a relationship set to describe an action that occurs between entities



For example, attribute date as attribute of advisor or as attribute of student

■ Placement of relationship attributes



Binary Vs. Non-Binary Relationships

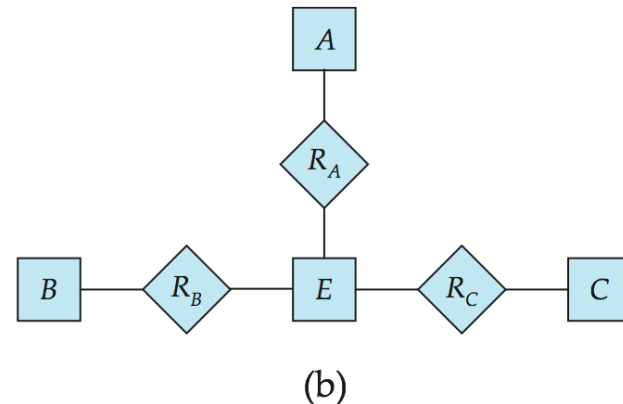
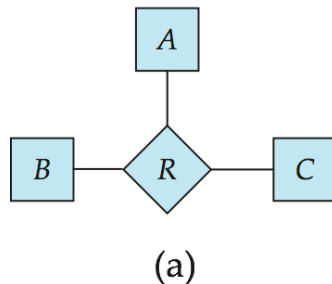
DataBase System Concepts

- Although it is possible to replace any non-binary (n -ary, for $n > 2$) relationship set by a number of distinct binary relationship sets, a n -ary relationship set shows more clearly that several entities participate in a single relationship.
- Some relationships that appear to be non-binary may be better represented using binary relationships
 - For example, a ternary relationship *parents*, relating a child to his/her father and mother, is best replaced by two binary relationships, *father* and *mother*
 - ▶ Using two binary relationships allows partial information (e.g., only mother being known)
 - But there are some relationships that are naturally non-binary
 - ▶ Example: *proj_guide*



Converting Non-Binary Relationships to Binary Form

- In general, any non-binary relationship can be represented using binary relationships by creating an artificial entity set.
 - Replace R between entity sets A , B and C by an entity set E , and three relationship sets:
 1. R_A , relating E and A
 2. R_B , relating E and B
 3. R_C , relating E and C
 - Create an identifying attribute for E and add any attributes of R to E
 - For each relationship (a_i, b_i, c_i) in R , create
 1. a new entity e_i in the entity set E
 2. add (e_i, a_i) to R_A
 3. add (e_i, b_i) to R_B
 4. add (e_i, c_i) to R_C



● *Converting Non-Binary Relationships (Cont.)*

DataBase System Concepts

■ Also need to translate constraints

- Translating all constraints may not be possible
- There may be instances in the translated schema that cannot correspond to any instance of R
 - ▶ Exercise: *add constraints to the relationships R_A , R_B and R_C to ensure that a newly created entity corresponds to exactly one entity in each of entity sets A , B and C*
- We can avoid creating an identifying attribute by making E a weak entity set (described shortly) identified by the three relationship sets

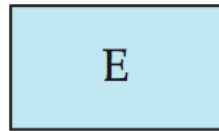


E-R Design Decisions

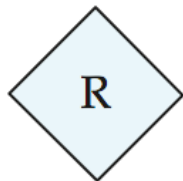
- The use of an attribute or entity set to represent an object.
- Whether a real-world concept is best expressed by an entity set or a relationship set.
- The use of a ternary relationship versus a pair of binary relationships.
- The use of a strong or weak entity set.
- The use of specialization/generalization – contributes to modularity in the design.
- The use of aggregation – can treat the aggregate entity set as a single unit without concern for the details of its internal structure.



Summary of Symbols Used in E-R Notation



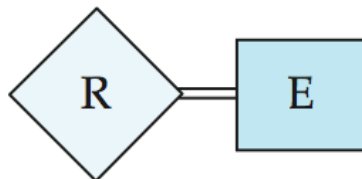
entity set



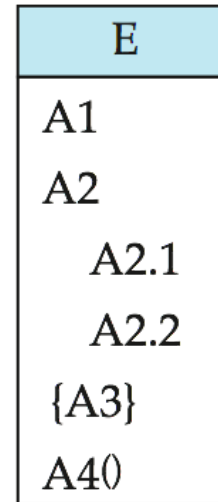
relationship set



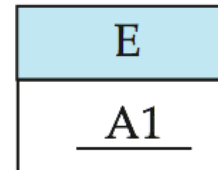
identifying
relationship set
for weak entity set



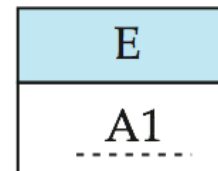
total participation
of entity set in
relationship



attributes:
simple (A1),
composite (A2) and
multivalued (A3)
derived (A4)

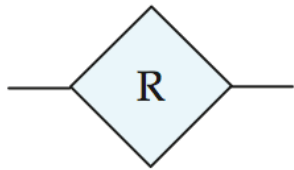


primary key

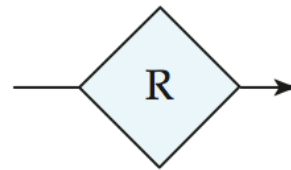


discriminating
attribute of
weak entity set

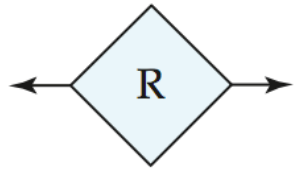
Symbols Used in E-R Notation (Cont.)



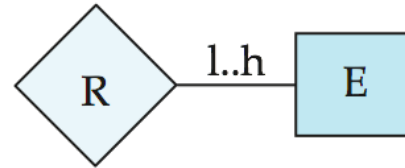
many-to-many
relationship



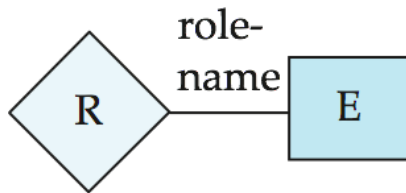
many-to-one
relationship



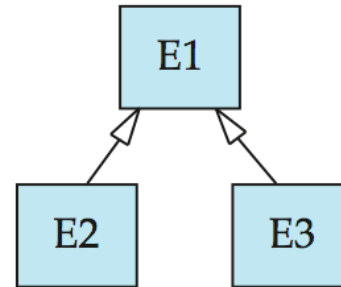
one-to-one
relationship



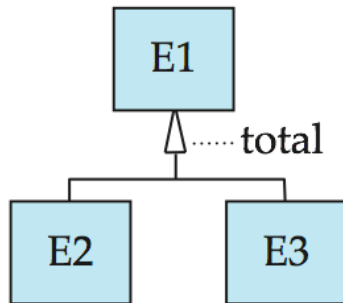
cardinality
limits



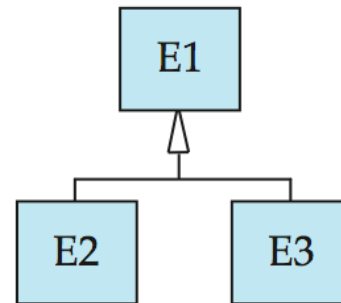
role indicator



ISA: generalization
or specialization



total (disjoint)
generalization

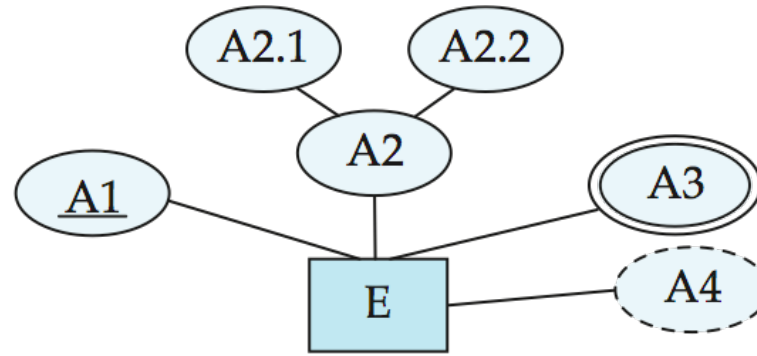


disjoint
generalization

Alternative ER Notations

■ Chen, IDE1FX, ...

entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



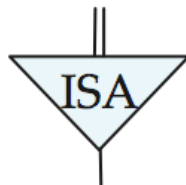
weak entity set



generalization



total
generalization

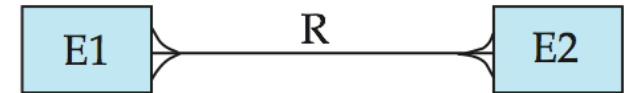
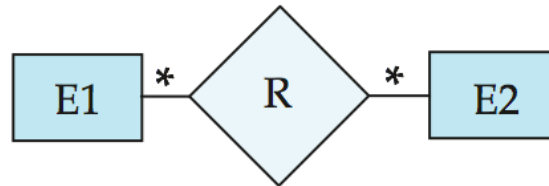


Alternative ER Notations

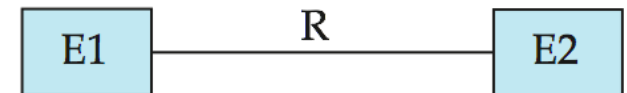
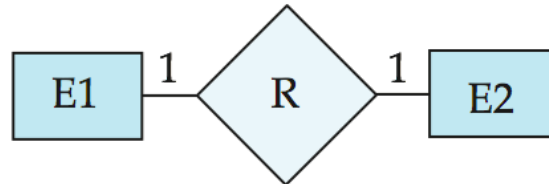
Chen

IDE1FX (Crows feet notation)

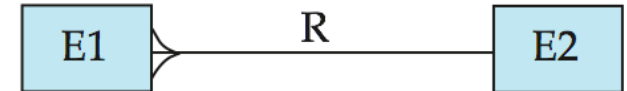
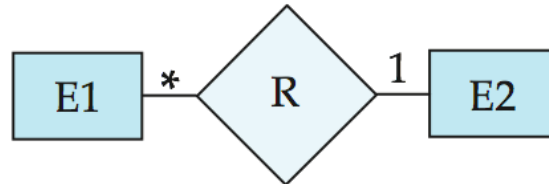
many-to-many
relationship



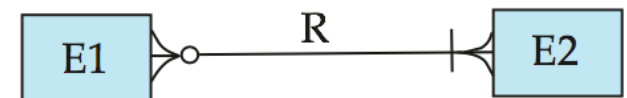
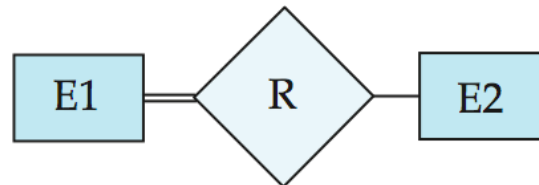
one-to-one
relationship



many-to-one
relationship



participation
in R: total (E1)
and partial (E2)

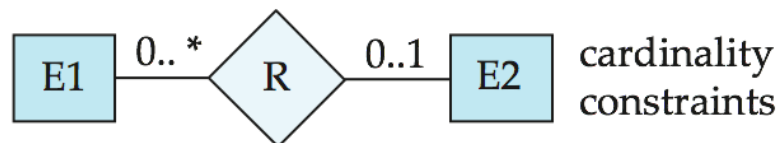
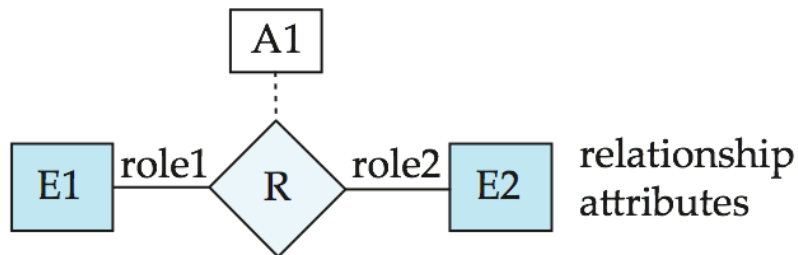
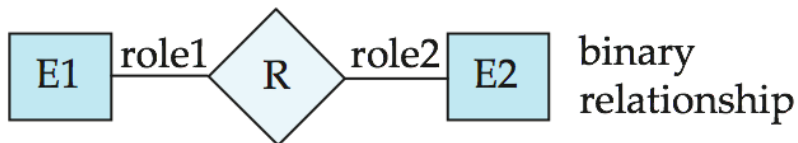
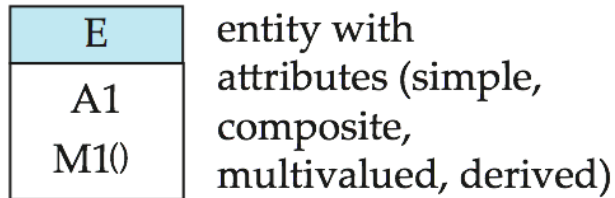


UML

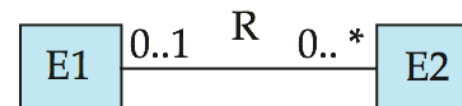
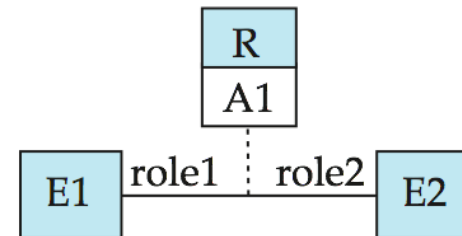
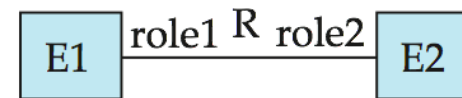
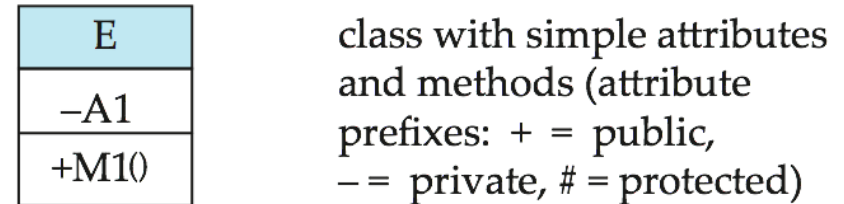
- **UML**: Unified Modeling Language
- UML has many components to graphically model different aspects of an entire software system
- UML Class Diagrams correspond to E-R Diagram, but several differences.

ER vs. UML Class Diagrams

ER Diagram Notation



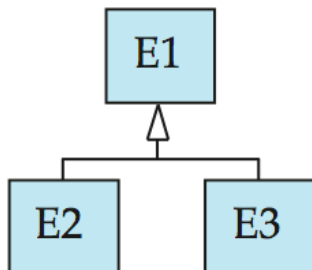
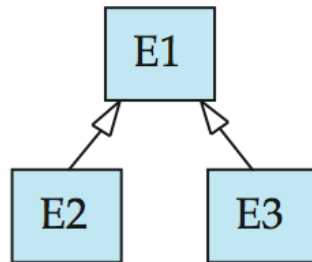
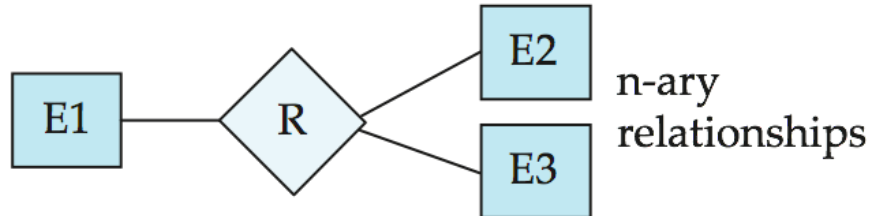
Equivalent in UML



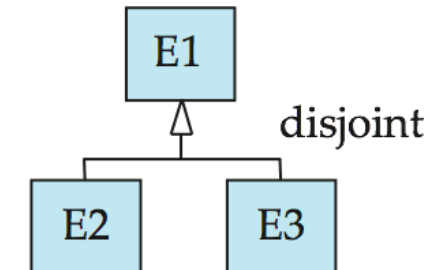
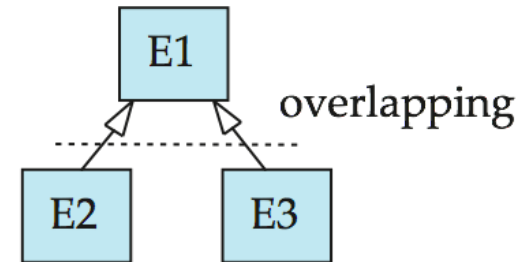
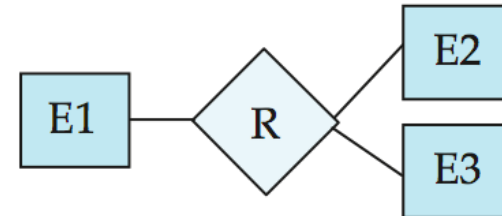
*Note reversal of position in cardinality constraint depiction

ER vs. UML Class Diagrams

ER Diagram Notation



Equivalent in UML



*Generalization can use merged or separate arrows independent of disjoint/overlapping

UML Class Diagrams (Cont.)

DataBase System Concepts

- Binary relationship sets are represented in UML by just drawing a line connecting the entity sets. The relationship set name is written adjacent to the line.
- The role played by an entity set in a relationship set may also be specified by writing the role name on the line, adjacent to the entity set.
- The relationship set name may alternatively be written in a box, along with attributes of the relationship set, and the box is connected, using a dotted line, to the line depicting the relationship set.



Reduction to Relation Schemas

Reduction to Relation Schemas

- Entity sets and relationship sets can be expressed uniformly as *relation schemas* that represent the contents of the database.
- A database which conforms to an E-R diagram can be represented by a collection of schemas.
- For each entity set and relationship set there is a unique schema that is assigned the name of the corresponding entity set or relationship set.
- Each schema has a number of columns (generally corresponding to attributes), which have unique names.

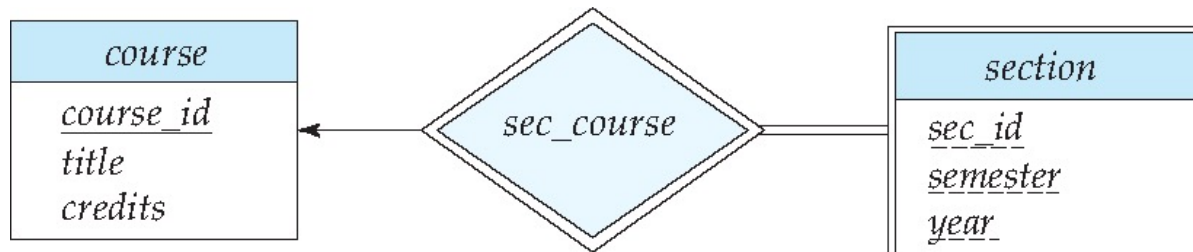


- A strong entity set reduces to a schema with the same attributes

student(ID, name, tot_cred)

- A weak entity set becomes a table that includes a column for the primary key of the identifying strong entity set

section (course id, sec id, sem, year)



Representation of Entity Sets with Composite Attributes

- Composite attributes are flattened out by creating a separate attribute for each component attribute

- Example: given entity set *instructor* with composite attribute *name* with component attributes *first_name* and *last_name* the schema corresponding to the entity set has two attributes *name_first_name* and *name_last_name*
 - ▶ Prefix omitted if there is no ambiguity (*name_first_name* could be *first_name*)

- Ignoring multivalued attributes, extended instructor schema is

- *instructor*(*ID*,
first_name, *middle_initial*, *last_name*,
street_number, *street_name*,
apt_number, *city*, *state*, *zip_code*,
date_of_birth)

<i>instructor</i>
<u><i>ID</i></u>
<i>name</i>
<i>first_name</i>
<i>middle_initial</i>
<i>last_name</i>
<i>address</i>
<i>street</i>
<i>street_number</i>
<i>street_name</i>
<i>apt_number</i>
<i>city</i>
<i>state</i>
<i>zip</i>
{ <i>phone_number</i> }
<i>date_of_birth</i>
<i>age</i> ()

Representation of Entity Sets with Multivalued Attributes

DataBase System Concepts

- A multivalued attribute M of an entity E is represented by a separate schema EM
- Schema EM has attributes corresponding to the primary key of E and an attribute corresponding to multivalued attribute M
- Example: Multivalued attribute *phone_number* of *instructor* is represented by a schema:
 $inst_phone = (\underline{ID}, \underline{phone_number})$
- Each value of the multivalued attribute maps to a separate tuple of the relation on schema EM
 - For example, an *instructor* entity with primary key 22222 and phone numbers 456-7890 and 123-4567 maps to two tuples:
(22222, 456-7890) and (22222, 123-4567)

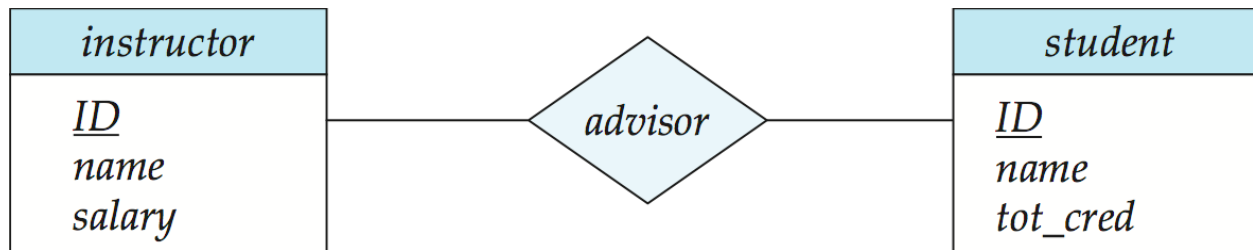


Representing Relationship Sets

DataBase System Concepts

- A many-to-many relationship set is represented as a schema with attributes for the primary keys of the two participating entity sets, and any descriptive attributes of the relationship set.
- Example: schema for relationship set *advisor*

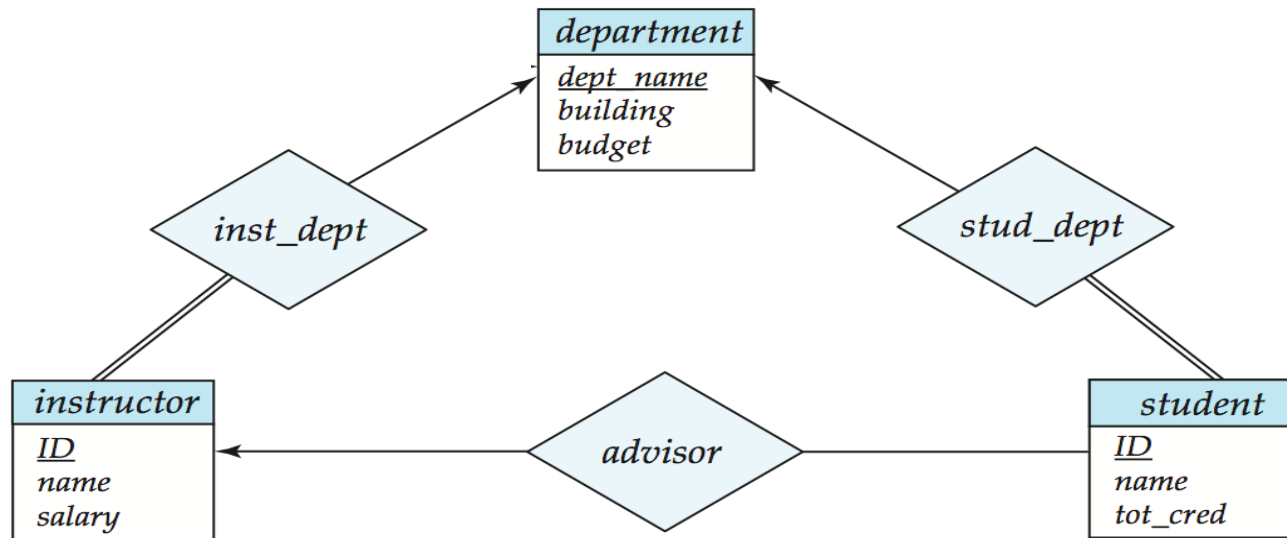
$$advisor = (\underline{s_id}, \underline{i_id})$$



Representing Relationship Sets

DataBase System Concepts

- Many-to-one and one-to-many relationship sets that are total on the many-side can be represented by adding an extra attribute to the “many” side, containing the primary key of the “one” side
- Example: Instead of creating a schema for relationship set *inst_dept*, add an attribute *dept_name* to the schema arising from entity set *instructor*

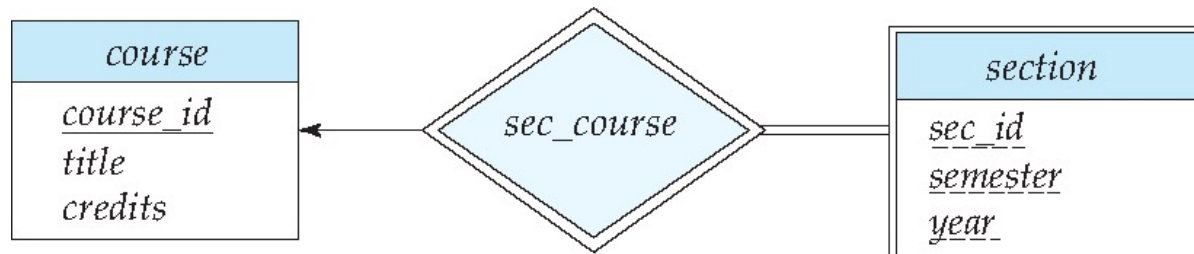


- For one-to-one relationship sets, either side can be chosen to act as the “many” side
 - That is, an extra attribute can be added to either of the tables corresponding to the two entity sets
- If participation is *partial* on the “many” side, replacing a schema by an extra attribute in the schema corresponding to the “many” side could result in null values



Redundancy of Schemas

- The schema corresponding to a relationship set linking a weak entity set to its identifying strong entity set is redundant.
- Example: The *section* schema already contains the attributes that would appear in the *sec_course* schema



Representing Specialization via Schemas

DataBase System Concepts

■ Method 1:

- Form a schema for the higher-level entity
- Form a schema for each lower-level entity set, include primary key of higher-level entity set and local attributes

schema	attributes
person	ID, name, street, city
student	ID, tot_cred
employee	ID, salary

- Drawback: getting information about, an *employee* requires accessing two relations, the one corresponding to the low-level schema and the one corresponding to the high-level schema



Representing Specialization as Schemas (Cont.)

DataBase System Concepts

Method 2:

- Form a schema for each entity set with all local and inherited attributes

schema	attributes
person	ID, name, street, city
student	ID, name, street, city, tot_cred
employee	ID, name, street, city, salary

- Drawback: *name*, *street* and *city* may be stored redundantly for people who are both students and employees



● *Representing Aggregation via Schemas*

DataBase System Concepts

- - To represent aggregation, create a schema containing
 - Primary key of the aggregated relationship,
 - The primary key of the associated entity set
 - Any descriptive attributes
 - In our example:
 - The schema *eval_for* is:
eval_for (*s_ID*, *project_id*, *i_ID*, *evaluation_id*)
 - The schema *proj_guide* is redundant.



End of Chapter 6